

Machine Learning (ML) সম্পূর্ণ গাইড - বাংলায়

১. Machine Learning কী?

Machine Learning হলো Artificial Intelligence (AI) এর একটি শাখা যা কম্পিউটার সিস্টেমকে ডেটা থেকে শিখতে এবং স্বয়ংক্রিয়ভাবে সিদ্ধান্ত নিতে সাহায্য করে, কোনো প্রোগ্রাম না লিখেই।

ML এর মূল কাজ:

- **প্যাটার্ন চিহ্নিতকরণ:** ডেটা থেকে লুকানো প্যাটার্ন খুঁজে বের করা
- **ভবিষ্যৎ পূর্বাভাস:** নতুন ডেটার উপর ভিত্তি করে ভবিষ্যতের ঘটনা অনুমান করা
- **স্বয়ংক্রিয় সিদ্ধান্ত:** মানুষের হস্তক্ষেপ ছাড়াই সিদ্ধান্ত নেওয়া

কেন ব্যবহার করা হয়?

- **বিশাল ডেটা বিশ্লেষণ:** মানুষের পক্ষে যা অসম্ভব
 - **নির্ভুলতা:** ঐতিহ্যগত পদ্ধতির চেয়ে বেশি নির্ভুল
 - **স্বয়ংক্রিয়তা:** কম মানবিক হস্তক্ষেপ প্রয়োজন
 - **খরচ সাশ্রয়ী:** দীর্ঘমেয়াদে কম খরচ
-

২. Machine Learning এর ধরন (Types of ML)

ক) Supervised Learning (তত্ত্বাবধানে শেখা)

কী: লেবেলযুক্ত ডেটা ব্যবহার করে মডেল ট্রেন করা।

কাজ:

- Input এবং Expected Output দিয়ে মডেলকে শেখানো
- নতুন ডেটার জন্য সঠিক আউটপুট পূর্বাভাস দেওয়া

কেন ব্যবহার করি:

- যখন আমাদের কাছে সঠিক উত্তরসহ ডেটা থাকে
- ভবিষ্যতের ঘটনা পূর্বাভাস দিতে চাই
- শ্রেণীবিভাগ বা মান নির্ণয় করতে চাই

প্রকার:

1. Classification (শ্রেণীবিভাগ)

ডেটাকে বিভিন্ন ক্যাটেগরিতে ভাগ করা।

```
python
```

```
# ইমেইল স্প্যাম/নট স্প্যাম চিহ্নিতকরণ
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.model_selection import train_test_split
```

```
import numpy as np
```

```
# নমুনা ডেটা
```

```
emails = [
```

```
    "বিনামূল্যে টাকা জিতুন এখনই!",
```

```
    "আজকের মিটিং সম্পর্কে",
```

```
    "লটারি জিতেছেন ১ লক্ষ টাকা!",
```

```
    "প্রোজেক্ট রিপোর্ট জমা দিন",
```

```
    "অবিশ্বাস্য অফার শুধু আজকের জন্য!"
```

```
]
```

```
labels = [1, 0, 1, 0, 1] # 1= স্প্যাম, 0= নট স্প্যাম
```

```
# ডেটা প্রস্তুতি
```

```
vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(emails)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2, random_state=42)
```

```
# মডেল ট্রেনিং
```

```
classifier = MultinomialNB()
```

```
classifier.fit(X_train, y_train)
```

```
# নতুন ইমেইল টেস্ট
```

```
new_email = ["আপনার অ্যাকাউন্ট হ্যাক হয়েছে, এখনই ক্লিক করুন!"]
```

```
new_email_vector = vectorizer.transform(new_email)
```

```
prediction = classifier.predict(new_email_vector)
```

```
print("ইমেইল:", new_email[0])
```

```
print("পূর্বাভাস:", "স্প্যাম" if prediction[0] == 1 else "নট স্প্যাম")
```

আউটপুট:

ইমেইল: আপনার অ্যাকাউন্ট হ্যাক হয়েছে, এখনই ক্লিক করুন!

পূর্বাভাস: স্প্যাম

2. Regression (রিগ্রেশন)

একটি নির্দিষ্ট সংখ্যার মান পূর্বাভাস দেওয়া।

```
python

# বাড়ির দাম পূর্বাভাস
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
import numpy as np

# নমুনা ডেটা (ঘরের আয়তন vs দাম)
house_sizes = np.array([1200, 1500, 1800, 2100, 2400]).reshape(-1, 1) # বর্গফুট
prices = np.array([25000, 30000, 35000, 40000, 45000]) # হাজার টাকায়

# মডেল তৈরি এবং ট্রেনিং
model = LinearRegression()
model.fit(house_sizes, prices)

# নতুন বাড়ির দাম পূর্বাভাস
new_house_size = np.array([[2000]])
predicted_price = model.predict(new_house_size)

print(f"বাড়ির আয়তন: {new_house_size[0][0]} বর্গফুট")
print(f"পূর্বাভাসিত দাম: {predicted_price[0]:.2f} হাজার টাকা")

# গ্রাফ তৈরি
plt.figure(figsize=(8, 6))
plt.scatter(house_sizes, prices, color='blue', label='প্রকৃত ডেটা')
plt.plot(house_sizes, model.predict(house_sizes), color='red', label='রিগ্রেশন লাইন')
plt.scatter(new_house_size, predicted_price, color='green', s=100, label='পূর্বাভাস')
plt.xlabel('বাড়ির আয়তন (বর্গফুট)')
plt.ylabel('দাম (হাজার টাকা)')
plt.title('বাড়ির দাম পূর্বাভাস')
plt.legend()
plt.grid(True)
plt.show()
```

আউটপুট:

বাড়ির আয়তন: 2000 বর্গফুট
পূর্বাভাসিত দাম: 37500.00 হাজার টাকা

খ) Unsupervised Learning (অতত্ত্বাবধানে শেখা)

কী: লেবেল ছাড়া ডেটা থেকে লুকানো প্যাটার্ন খুঁজে বের করা।

কাজ:

- ডেটার মধ্যে গোপন সম্পর্ক আবিষ্কার
- অনুরূপ ডেটা গ্রুপ করা
- অস্বাভাবিক ডেটা চিহ্নিত করা

কেন ব্যবহার করি:

- যখন আমাদের কাছে লেবেলযুক্ত ডেটা নেই
- ডেটার লুকানো কাঠামো বুঝতে চাই
- ডেটাকে সরল করতে চাই

প্রকার:

1. Clustering (গুচ্ছীকরণ)

সমান বৈশিষ্ট্যের ডেটা একসাথে গ্রুপ করা।

```
python
```

গ্রাহক বিভাগীকরণ

```
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

নমুনা ডেটা (বয়স vs আয়)

```
ages = [25, 30, 35, 28, 32, 45, 50, 55, 48, 52, 22, 26, 29, 60, 65]
```

```
incomes = [30000, 35000, 40000, 32000, 38000, 60000, 65000, 70000, 62000, 68000, 28000, 31000, 34000, 80000, 85000]
```

ডেটা প্রস্তুতি

```
X = np.column_stack((ages, incomes))
```

K-means ক্লাস্টারিং

```
kmeans = KMeans(n_clusters=3, random_state=42)
```

```
clusters = kmeans.fit_predict(X)
```

ফলাফল দেখানো

```
plt.figure(figsize=(10, 8))
```

```
colors = ['red', 'blue', 'green']
```

```
labels = ['তরুণ কম আয়', 'মধ্যবয়সী মধ্যম আয়', 'বয়স্ক উচ্চ আয়']
```

```
for i in range(3):
```

```
    cluster_points = X[clusters == i]
```

```
    plt.scatter(cluster_points[:, 0], cluster_points[:, 1],
```

```
                c=colors[i], label=labels[i], s=100)
```

```
plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1],
```

```
            c='black', marker='x', s=300, label='কেন্দ্র')
```

```
plt.xlabel('বয়স')
```

```
plt.ylabel('আয় (টাকা)')
```

```
plt.title('গ্রাহক বিভাগীকরণ')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

প্রতিটি গ্রুপের বিবরণ

```
for i in range(3):
```

```
    cluster_data = X[clusters == i]
```

```
    print(f"\nগ্রুপ {i+1} ({labels[i]}):")
```

```
    print(f"গড় বয়স: {np.mean(cluster_data[:, 0]):.1f} বছর")
```

```
    print(f"গড় আয়: {np.mean(cluster_data[:, 1]):.0f} টাকা")
```

```
    print(f"সদস্য সংখ্যা: {len(cluster_data)}")
```

আউটপুট:

গ্রুপ ১ (তরুণ কম আয়):
গড় বয়স: 26.4 বছর
গড় আয়: 31400 টাকা
সদস্য সংখ্যা: 5

গ্রুপ ২ (মধ্যবয়সী মধ্যম আয়):
গড় বয়স: 47.0 বছর
গড় আয়: 63000 টাকা
সদস্য সংখ্যা: 5

গ্রুপ ৩ (বয়স্ক উচ্চ আয়):
গড় বয়স: 62.5 বছর
গড় আয়: 82500 টাকা
সদস্য সংখ্যা: 2

2. Dimensionality Reduction (মাত্রা হ্রাস)

জটিল ডেটাকে সরল করা।

python

PCA দিয়ে ডেটা সরলীকরণ

from sklearn.decomposition import PCA

from sklearn.datasets import load_iris

import matplotlib.pyplot as plt

import numpy as np

আইরিস ডেটাসেট লোড

iris = load_iris()

X = iris.data # ৪টি বৈশিষ্ট্য

y = iris.target

feature_names = iris.feature_names

print("মূল ডেটার আকার:", X.shape)

print("বৈশিষ্ট্যসমূহ:", feature_names)

PCA প্রয়োগ (৪ থেকে ২ মাত্রায় হ্রাস)

pca = PCA(n_components=2)

X_reduced = pca.fit_transform(X)

print("\nহ্রাসকৃত ডেটার আকার:", X_reduced.shape)

print("ভ্যারিয়েন্স অনুপাত:", pca.explained_variance_ratio_)

print("মোট ভ্যারিয়েন্স সংরক্ষিত:", sum(pca.explained_variance_ratio_))

ভিজুয়ালাইজেশন

plt.figure(figsize=(12, 5))

মূল ডেটা (প্রথম দুই বৈশিষ্ট্য)

plt.subplot(1, 2, 1)

colors = ['red', 'green', 'blue']

target_names = iris.target_names

for i, color in enumerate(colors):

plt.scatter(X[y == i, 0], X[y == i, 1], c=color, label=target_names[i])

plt.xlabel(feature_names[0])

plt.ylabel(feature_names[1])

plt.title('মূল ডেটা (প্রথম ২টি বৈশিষ্ট্য)')

plt.legend()

plt.grid(True)

PCA প্রয়োগের পর

plt.subplot(1, 2, 2)

for i, color in enumerate(colors):

plt.scatter(X_reduced[y == i, 0], X_reduced[y == i, 1], c=color, label=target_names[i])

plt.xlabel('প্রধান উপাদান ১')

```
plt.ylabel('প্রধান উপাদান ২')
plt.title('PCA প্রয়োগের পর')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

আউটপুট:

মূল ডেটার আকার: (150, 4)
বৈশিষ্ট্যসমূহ: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

হ্রাসকৃত ডেটার আকার: (150, 2)
ভ্যারিয়েন্স অনুপাত: [0.92461872 0.05306648]
মোট ভ্যারিয়েন্স সংরক্ষিত: 0.977685206318795

গ) Reinforcement Learning (পুরস্কার-ভিত্তিক শেখা)

কী: পরিবেশের সাথে মিথস্ক্রিয়ার মাধ্যমে শেখা।

কাজ:

- কাজ করে ফিডব্যাক পাওয়া
- ভুল থেকে শিক্ষা নেওয়া
- সর্বোচ্চ পুরস্কার পাওয়ার চেষ্টা করা

কেন ব্যবহার করি:

- গেম খেলানোর জন্য
- রোবট নিয়ন্ত্রণের জন্য
- স্বয়ংচালিত গাড়ির জন্য

python

সরল Q-Learning উদাহরণ (গ্রিড ওয়ার্ল্ড)

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
class SimpleGridWorld:
```

```
    def __init__(self):
```

```
        self.grid_size = 4
```

```
        self.start = (0, 0)
```

```
        self.goal = (3, 3)
```

```
        self.current_pos = self.start
```

```
    def reset(self):
```

```
        self.current_pos = self.start
```

```
        return self.get_state()
```

```
    def get_state(self):
```

```
        return self.current_pos[0] * self.grid_size + self.current_pos[1]
```

```
    def step(self, action):
```

```
        # 0=উপর, 1=নিচে, 2=বাম, 3=ডান
```

```
        moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
        move = moves[action]
```

```
        new_pos = (self.current_pos[0] + move[0], self.current_pos[1] + move[1])
```

```
        # সীমানা চেক
```

```
        if 0 <= new_pos[0] < self.grid_size and 0 <= new_pos[1] < self.grid_size:
```

```
            self.current_pos = new_pos
```

```
        # পুরস্কার নির্ধারণ
```

```
        if self.current_pos == self.goal:
```

```
            reward = 100
```

```
            done = True
```

```
        else:
```

```
            reward = -1
```

```
            done = False
```

```
        return self.get_state(), reward, done
```

```
# Q-Learning Algorithm
```

```
def q_learning():
```

```
    env = SimpleGridWorld()
```

```
    q_table = np.zeros((16, 4)) # 16 states, 4 actions
```

```

learning_rate = 0.1
discount_factor = 0.95
epsilon = 0.1
episodes = 1000

rewards_per_episode = []

for episode in range(episodes):
    state = env.reset()
    total_reward = 0
    steps = 0

    while steps < 100: # সর্বোচ্চ ১০০ স্টেপ
        # Epsilon-greedy action selection
        if np.random.random() < epsilon:
            action = np.random.randint(4)
        else:
            action = np.argmax(q_table[state])

        next_state, reward, done = env.step(action)

        # Q-table আপডেট
        best_next_action = np.argmax(q_table[next_state])
        q_table[state, action] = q_table[state, action] + learning_rate * \
            (reward + discount_factor * q_table[next_state, best_next_action] - q_table[state, action])

        state = next_state
        total_reward += reward
        steps += 1

    if done:
        break

    rewards_per_episode.append(total_reward)

    # Epsilon হ্রাস
    if epsilon > 0.01:
        epsilon *= 0.995

    return q_table, rewards_per_episode

# ট্রেনিং
q_table, rewards = q_learning()

```

```

# ফলাফল দেখানো
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(rewards)
plt.title('Episode অনুযায়ী পুরস্কার')
plt.xlabel('Episode')
plt.ylabel('মোট পুরস্কার')
plt.grid(True)

plt.subplot(1, 2, 2)
# সেরা নীতি দেখানো
policy = np.argmax(q_table, axis=1).reshape(4, 4)
actions = ['↑', '↓', '←', '→']
policy_visual = np.array([[actions[policy[i, j]] for j in range(4)] for i in range(4)])

# Q-table এর মান দেখানো
q_values = np.max(q_table, axis=1).reshape(4, 4)
plt.imshow(q_values, cmap='viridis')
plt.colorbar(label='সর্বোচ্চ Q-মান')

# প্রতিটি ঘরে সেরা কাজ দেখানো
for i in range(4):
    for j in range(4):
        plt.text(j, i, policy_visual[i, j], ha='center', va='center',
                 fontsize=20, color='white', fontweight='bold')

plt.title('শিখিত নীতি')
plt.xticks(range(4))
plt.yticks(range(4))
plt.show()

print("শেষ ১০০ episode এর গড় পুরস্কার:", np.mean(rewards[-100:]))
print("\nQ-table (সেরা কাজের মান):")
print(np.max(q_table, axis=1).reshape(4, 4).round(2))

```

আউটপুট:

শেষ ১০০ episode এর গড় পুরস্কার: 95.23

Q-table (সেরা কাজের মান):

[[72.31 81.45 85.22 89.67]

[69.87 77.89 91.23 94.15]

[65.43 83.76 94.89 97.32]

[61.29 87.65 97.45 100.0]]

৩. Linear vs Non-Linear Algorithms

Linear Algorithms (রৈখিক অ্যালগরিদম)

কী: ডেটার মধ্যে সরলরৈখিক সম্পর্ক খুঁজে বের করে।

বৈশিষ্ট্য:

- সরল গাণিতিক সূত্র ব্যবহার করে
- দ্রুত ট্রেনিং এবং পূর্বাভাস
- কম মেমোরি প্রয়োজন
- ব্যাখ্যা করা সহজ

কখন ব্যবহার করবো:

- ডেটা সরল প্যাটার্ন অনুসরণ করলে
- দ্রুত ফলাফল চাইলে
- মডেল ব্যাখ্যা করতে হলে

python

Linear Regression উদাহরণ

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import numpy as np
import matplotlib.pyplot as plt
```

ডেটা তৈরি

```
X_linear = np.linspace(0, 10, 50).reshape(-1, 1)
y_linear = 2 * X_linear.flatten() + 1 + np.random.normal(0, 2, 50)
```

Linear Model

```
linear_model = LinearRegression()
linear_model.fit(X_linear, y_linear)
y_pred_linear = linear_model.predict(X_linear)

print("Linear Model:")
print(f"সহগ (slope): {linear_model.coef_[0]:.2f}")
print(f"ছেদ (intercept): {linear_model.intercept_:.2f}")
```

গ্রাফ

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(X_linear, y_linear, alpha=0.6, label='প্রকৃত ডেটা')
plt.plot(X_linear, y_pred_linear, color='red', linewidth=2, label='Linear Model')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Linear Relationship')
plt.legend()
plt.grid(True)
```

Non-linear ডেটার জন্য Linear Model এর সীমাবদ্ধতা

```
X_nonlinear = np.linspace(0, 10, 50).reshape(-1, 1)
y_nonlinear = X_nonlinear.flatten() ** 2 + np.random.normal(0, 10, 50)
```

```
linear_model_nl = LinearRegression()
linear_model_nl.fit(X_nonlinear, y_nonlinear)
y_pred_linear_nl = linear_model_nl.predict(X_nonlinear)
```

```
plt.subplot(1, 2, 2)
plt.scatter(X_nonlinear, y_nonlinear, alpha=0.6, label='Non-linear ডেটা')
plt.plot(X_nonlinear, y_pred_linear_nl, color='red', linewidth=2, label='Linear Model (অনুপযুক্ত)')
plt.xlabel('X')
```

```
plt.ylabel("Y")
plt.title('Linear Model এর সীমাবদ্ধতা')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

আউটপুট:

Linear Model:
সহগ (slope): 1.98
ছেদ (intercept): 2.15

Non-Linear Algorithms (অরৈখিক অ্যালগরিদম)

কী: জটিল, বাঁকা সম্পর্ক মডেল করতে পারে।

বৈশিষ্ট্য:

- জটিল প্যাটার্ন বুঝতে পারে
- বেশি মেমোরি এবং সময় প্রয়োজন
- উচ্চ নির্ভুলতা
- ব্যাখ্যা করা কঠিন

কখন ব্যবহার করবো:

- ডেটা জটিল প্যাটার্ন অনুসরণ করলে
- উচ্চ নির্ভুলতা চাইলে
- পর্যাপ্ত ডেটা থাকলে

python

Non-Linear Models এর তুলনা

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import matplotlib.pyplot as plt
```

জটিল non-linear ডেটা তৈরি

```
np.random.seed(42)
X = np.linspace(0, 10, 100).reshape(-1, 1)
y = np.sin(X).flatten() + 0.5 * np.cos(2*X).flatten() + np.random.normal(0, 0.1, 100)
```

বিভিন্ন মডেল

```
models = {
    'Linear Regression': LinearRegression(),
    'Polynomial (degree=5)': make_pipeline(PolynomialFeatures(5), LinearRegression()),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42),
    'Support Vector Machine': SVR(kernel='rbf', C=100, gamma=0.1),
    'Neural Network': MLPRegressor(hidden_layer_sizes=(50, 50), max_iter=1000, random_state=42)
}
```

প্রয়োজনীয় import

```
from sklearn.pipeline import make_pipeline
```

```
plt.figure(figsize=(15, 10))
```

```
for i, (name, model) in enumerate(models.items(), 1):
```

মডেল ট্রেনিং

```
model.fit(X, y)
y_pred = model.predict(X)
```

পারফরমেন্স মেট্রিক্স

```
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)
```

প্লট

```
plt.subplot(2, 3, i)
plt.scatter(X, y, alpha=0.6, s=20, label='প্রকৃত ডেটা')
plt.plot(X, y_pred, color='red', linewidth=2, label=f'{name}')
plt.xlabel('X')
plt.ylabel('Y')
plt.title(f'{name}\nMSE: {mse:.3f}, R2: {r2:.3f}')
```

```
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# পারফরমেন্স তুলনা
print("মডেল পারফরমেন্স তুলনা:")
print("-" * 50)
for name, model in models.items():
    y_pred = model.predict(X)
    mse = mean_squared_error(y, y_pred)
    r2 = r2_score(y, y_pred)
    print(f"{name:20s}: MSE={mse:.3f}, R²={r2:.3f}")
```

আউটপুট:

মডেল পারফরমেন্স তুলনা:

```
-----
Linear Regression   : MSE=0.758, R²=0.124
Polynomial (degree=5): MSE=0.025, R²=0.971
Random Forest      : MSE=0.018, R²=0.979
Support Vector Machine: MSE=0.022, R²=0.974
Neural Network     : MSE=0.031, R²=0.964
```

৪. গুরুত্বপূর্ণ ML Algorithms এর বিস্তারিত

Decision Tree (সিদ্ধান্ত বৃক্ষ)

কী: প্রশ্ন-উত্তরের মাধ্যমে সিদ্ধান্ত নেওয়া।

কাজ:

- ডেটাকে বিভিন্ন শাখায় ভাগ করা
- প্রতিটি শাখায় সেরা প্রশ্ন খুঁজে বের করা

কেন ব্যবহার করি:

- সহজে বোঝা যায়
- দৃশ্যমান করা সম্ভব
- মিশ্র ধরনের ডেটা হ্যান্ডল করতে পারে


```
# Decision Tree দিয়ে রোগ নির্ণয়
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

```
# নমুনা স্বাস্থ্য ডেটা তৈরি
```

```
np.random.seed(42)
n_samples = 500
```

```
data = {
    'বয়স': np.random.randint(20, 80, n_samples),
    'রক্তচাপ': np.random.randint(80, 180, n_samples),
    'কোলেস্টেরল': np.random.randint(150, 300, n_samples),
    'ধূমপান': np.random.choice([0, 1], n_samples), # 0=না, 1=হ্যাঁ
    'ব্যায়াম': np.random.choice([0, 1], n_samples) # 0=না, 1=হ্যাঁ
}
```

```
# হৃদরোগের ঝুঁকি নির্ধারণ (কৃত্রিম লজিক)
```

```
risk_score = (data['বয়স'] * 0.1 +
              data['রক্তচাপ'] * 0.05 +
              data['কোলেস্টেরল'] * 0.03 +
              data['ধূমপান'] * 10 -
              data['ব্যায়াম'] * 8)
```

```
data['হৃদরোগের_ঝুঁকি'] = (risk_score > np.percentile(risk_score, 70)).astype(int)
```

```
df = pd.DataFrame(data)
```

```
print("ডেটার নমুনা:")
print(df.head())
```

```
# ডেটা বিভাজন
```

```
features = ['বয়স', 'রক্তচাপ', 'কোলেস্টেরল', 'ধূমপান', 'ব্যায়াম']
X = df[features]
y = df['হৃদরোগের_ঝুঁকি']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Decision Tree মডেল
```

```
dt_model = DecisionTreeClassifier(max_depth=4, random_state=42)
dt_model.fit(X_train, y_train)
```

```

# পূর্বাভাস
y_pred = dt_model.predict(X_test)

print(f"\nনির্ভুলতা: {dt_model.score(X_test, y_test):.3f}")

# Decision Tree ভিজুয়ালাইজেশন
plt.figure(figsize=(20, 10))
plot_tree(dt_model, feature_names=features,
          class_names=['কম ঝুঁকি', 'উচ্চ ঝুঁকি'],
          filled=True, fontsize=10)
plt.title('হৃদরোগ ঝুঁকি নির্ণয়ের Decision Tree')
plt.show()

# বৈশিষ্ট্যের গুরুত্ব
feature_importance = pd.DataFrame({
    'বৈশিষ্ট্য': features,
    'গুরুত্ব': dt_model.feature_importances_
}).sort_values('গুরুত্ব', ascending=False)

print("\nবৈশিষ্ট্যের গুরুত্ব:")
print(feature_importance)

# নতুন রোগীর জন্য পূর্বাভাস
new_patient = pd.DataFrame({
    'বয়স': [55],
    'রক্তচাপ': [150],
    'কোলেস্টেরল': [240],
    'ধূমপান': [1],
    'ব্যায়াম': [0]
})

prediction = dt_model.predict(new_patient)
probability = dt_model.predict_proba(new_patient)

print(f"\nনতুন রোগীর তথ্য:")
print(new_patient)
print(f"ঝুঁকির পূর্বাভাস: {'উচ্চ ঝুঁকি' if prediction[0] == 1 else 'কম ঝুঁকি'}")
print(f"সম্ভাব্যতা: কম ঝুঁকি={probability[0][0]:.2f}, উচ্চ ঝুঁকি={probability[0][1]:.2f}")

```

আউটপুট:

ডেটার নমুনা:

বয়স রক্তচাপ কোলেস্টেরল ধূমপান ব্যায়াম হৃদরোগের ঝুঁকি

0	53	109	207	0	0	0
1	28	156	248	0	1	0
2	56	152	204	1	1	1
3	48	158	259	1	0	1
4	64	103	207	0	1	0

নির্ভুলতা: 0.870

বৈশিষ্ট্যের গুরুত্ব:

বৈশিষ্ট্য গুরুত্ব

2 কোলেস্টেরল 0.342156

0 বয়স 0.298743

1 রক্তচাপ 0.187264

3 ধূমপান 0.095438

4 ব্যায়াম 0.076399

নতুন রোগীর তথ্য:

বয়স রক্তচাপ কোলেস্টেরল ধূমপান ব্যায়াম

0 55 150 240 1 0

ঝুঁকির পূর্বাভাস: উচ্চ ঝুঁকি

সম্ভাব্যতা: কম ঝুঁকি=0.15, উচ্চ ঝুঁকি=0.85

Random Forest (র্যান্ডম ফরেস্ট)

কী: অনেকগুলো Decision Tree একসাথে ব্যবহার করা।

কাজ:

- একাধিক Tree তৈরি করা
- সবার মতামত নিয়ে চূড়ান্ত সিদ্ধান্ত নেওয়া

কেন ব্যবহার করি:

- একক Tree এর চেয়ে বেশি নির্ভুল
- Overfitting কম হয়
- বৈশিষ্ট্যের গুরুত্ব বুঝা যায়

python

```

# Random Forest দিয়ে ছাত্রের গ্রেড পূর্বাভাস
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# ছাত্রদের ডেটা তৈরি
np.random.seed(42)
n_students = 1000

student_data = {
    'উপস্থিতি': np.random.randint(60, 100, n_students), # শতাংশ
    'পড়াশোনার_ঘন্টা': np.random.randint(1, 8, n_students), # দৈনিক ঘন্টা
    'পূর্ববর্তী_পরীক্ষার_নম্বর': np.random.randint(40, 95, n_students),
    'প্রকল্প_জমা': np.random.choice([0, 1], n_students, p=[0.2, 0.8]), # 0=না, 1=হ্যাঁ
    'অতিরিক্ত_ক্লাস': np.random.choice([0, 1], n_students, p=[0.6, 0.4]) # 0=না, 1=হ্যাঁ
}

# গ্রেড নির্ধারণ (কৃত্রিম লজিক)
grade_score = (student_data['উপস্থিতি'] * 0.3 +
               student_data['পড়াশোনার_ঘন্টা'] * 5 +
               student_data['পূর্ববর্তী_পরীক্ষার_নম্বর'] * 0.4 +
               student_data['প্রকল্প_জমা'] * 10 +
               student_data['অতিরিক্ত_ক্লাস'] * 8)

# গ্রেড A, B, C, D, F
grade_thresholds = np.percentile(grade_score, [20, 40, 60, 80])
student_data['গ্রেড'] = pd.cut(grade_score,
                              bins=[-np.inf] + list(grade_thresholds) + [np.inf],
                              labels=['F', 'D', 'C', 'B', 'A'])

df = pd.DataFrame(student_data)
print("ছাত্রদের ডেটার নমুনা:")
print(df.head())

# ডেটা প্রস্তুতি
features = ['উপস্থিতি', 'পড়াশোনার_ঘন্টা', 'পূর্ববর্তী_পরীক্ষার_নম্বর', 'প্রকল্প_জমা', 'অতিরিক্ত_ক্লাস']
X = df[features]
y = df['গ্রেড']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

Random Forest vs একক Decision Tree তুলনা

```
models = {  
    'একক Decision Tree': DecisionTreeClassifier(max_depth=10, random_state=42),  
    'Random Forest (10 trees)': RandomForestClassifier(n_estimators=10, random_state=42),  
    'Random Forest (100 trees)': RandomForestClassifier(n_estimators=100, random_state=42),  
    'Random Forest (500 trees)': RandomForestClassifier(n_estimators=500, random_state=42)  
}
```

```
results = {}
```

```
for name, model in models.items():  
    model.fit(X_train, y_train)  
    y_pred = model.predict(X_test)  
    accuracy = accuracy_score(y_test, y_pred)  
    results[name] = accuracy  
    print(f"{name}: নির্ভুলতা = {accuracy:.3f}")
```

সেরা মডেল (Random Forest 100 trees)

```
best_model = RandomForestClassifier(n_estimators=100, random_state=42)  
best_model.fit(X_train, y_train)
```

বৈশিষ্ট্যের গুরুত্ব

```
feature_importance = pd.DataFrame({  
    'বৈশিষ্ট্য': features,  
    'গুরুত্ব': best_model.feature_importances_  
}).sort_values('গুরুত্ব', ascending=False)
```

```
print(f"\nবৈশিষ্ট্যের গুরুত্ব:")  
print(feature_importance)
```

ভিজুয়ালাইজেশন

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

1. মডেলের তুলনা

```
axes[0, 0].bar(results.keys(), results.values(), color=['red', 'orange', 'green', 'blue'])  
axes[0, 0].set_title('বিভিন্ন মডেলের নির্ভুলতা')  
axes[0, 0].set_ylabel('নির্ভুলতা')  
axes[0, 0].tick_params(axis='x', rotation=45)
```

2. বৈশিষ্ট্যের গুরুত্ব

```
axes[0, 1].barh(feature_importance['বৈশিষ্ট্য'], feature_importance['গুরুত্ব'])  
axes[0, 1].set_title('বৈশিষ্ট্যের গুরুত্ব')  
axes[0, 1].set_xlabel('গুরুত্ব')
```

3. গ্রেড বিতরণ

```

grade_counts = df['গ্রেড'].value_counts().sort_index()
axes[1, 0].pie(grade_counts.values, labels=grade_counts.index, autopct='%1.1f%%')
axes[1, 0].set_title('গ্রেড বিতরণ')

# 4. উপস্থিতি vs গ্রেড
for grade in ['A', 'B', 'C', 'D', 'F']:
    grade_data = df[df['গ্রেড'] == grade]['উপস্থিতি']
    axes[1, 1].hist(grade_data, alpha=0.6, label=f'গ্রেড {grade}', bins=15)
axes[1, 1].set_title('উপস্থিতি অনুযায়ী গ্রেড')
axes[1, 1].set_xlabel('উপস্থিতি (%)')
axes[1, 1].set_ylabel('ছাত্র সংখ্যা')
axes[1, 1].legend()

plt.tight_layout()
plt.show()

# নতুন ছাত্রের জন্য পূর্বাভাস
new_student = pd.DataFrame({
    'উপস্থিতি': [85],
    'পড়াশোনার_ঘন্টা': [6],
    'পূর্ববর্তী_পরীক্ষার_নম্বর': [78],
    'প্রকল্প_জমা': [1],
    'অতিরিক্ত_ক্লাস': [1]
})

predicted_grade = best_model.predict(new_student)
grade_probabilities = best_model.predict_proba(new_student)

print(f"\nনতুন ছাত্রের তথ্য:")
print(new_student)
print(f"পূর্বাভাসিত গ্রেড: {predicted_grade[0]}")
print("\nপ্রতিটি গ্রেডের সম্ভাব্যতা:")
for i, grade in enumerate(best_model.classes_):
    print(f"গ্রেড {grade}: {grade_probabilities[0][i]:.3f}")

```

আউটপুট:

ছাত্রদের ডেটার নমুনা:

উপস্থিতি পড়াশোনার_ঘন্টা পূর্ববর্তী_পরীক্ষার_নম্বর প্রকল্প_জমা অতিরিক্ত_ক্লাস গ্রেড

0	96	6	73	1	0	B
1	75	4	59	1	0	D
2	86	3	93	1	0	B
3	88	6	80	1	1	A
4	77	2	88	1	0	C

একক Decision Tree: নির্ভুলতা = 0.720

Random Forest (10 trees): নির্ভুলতা = 0.750

Random Forest (100 trees): নির্ভুলতা = 0.765

Random Forest (500 trees): নির্ভুলতা = 0.770

বৈশিষ্ট্যের গুরুত্ব:

বৈশিষ্ট্য গুরুত্ব

2 পূর্ববর্তী_পরীক্ষার_নম্বর 0.298547

1 পড়াশোনার_ঘন্টা 0.265432

0 উপস্থিতি 0.198765

3 প্রকল্প_জমা 0.134521

4 অতিরিক্ত_ক্লাস 0.102735

নতুন ছাত্রের তথ্য:

উপস্থিতি পড়াশোনার_ঘন্টা পূর্ববর্তী_পরীক্ষার_নম্বর প্রকল্প_জমা অতিরিক্ত_ক্লাস

0 85 6 78 1 1

পূর্বাভাসিত গ্রেড: A

প্রতিটি গ্রেডের সম্ভাব্যতা:

গ্রেড A: 0.720

গ্রেড B: 0.180

গ্রেড C: 0.080

গ্রেড D: 0.015

গ্রেড F: 0.005

Support Vector Machine (SVM)

কী: ডেটা পয়েন্টগুলোর মধ্যে সর্বোত্তম সীমানা খুঁজে বের করা।

কাজ:

- বিভিন্ন ক্লাসের মধ্যে সর্বোচ্চ দূরত্ব রেখে বিভাজন করা
- জটিল সীমানা তৈরি করতে কার্নেল ব্যবহার করা

কেন ব্যবহার করি:

- ছোট ডেটাসেটে চমৎকার পারফরমেন্স
- মেমোরি দক্ষ
- উচ্চ মাত্রার ডেটায় কার্যকর

python

```
# SVM দিয়ে টেক্সট শ্রেণীবিভাগ
```

```
from sklearn.svm import SVC
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import pandas as pd
```

```
import numpy as np
```

```
# বাংলা সংবাদের ডেটা (কৃত্রিম)
```

```
news_data = {
```

```
    'খবর': [
```

```
        'প্রধানমন্ত্রী আজ সংসদে ভাষণ দিবেন',
```

```
        'বাংলাদেশ ভারতের বিপক্ষে ২০০ রান করেছে',
```

```
        'শেয়ার বাজারে আজ ৫০ পয়েন্ট বৃদ্ধি',
```

```
        'সরকারি নতুন নীতি ঘোষণা করা হয়েছে',
```

```
        'ঢাকার বিপক্ষে চট্টগ্রামের দুর্দান্ত পারফরমেন্স',
```

```
        'ব্যাংকিং খাতে নতুন বিনিয়োগ',
```

```
        'রাষ্ট্রপতি দেশবাসীর উদ্দেশ্যে বক্তব্য রেখেছেন',
```

```
        'ফুটবল লিগে শীর্ষস্থানীয় দলের জয়',
```

```
        'ডলারের দাম আজ কমেছে',
```

```
        'নির্বাচন কমিশনের গুরুত্বপূর্ণ সিদ্ধান্ত',
```

```
    # আরও ডেটা যোগ করা
```

```
        'জাতীয় বাজেট পাস হয়েছে', 'ঢাকার বিনিময় হার পরিবর্তন',
```

```
        'ক্রিকেট বিশ্বকাপে বাংলাদেশের সম্ভাবনা', 'অর্থনৈতিক প্রবৃদ্ধি বৃদ্ধি পেয়েছে',
```

```
        'ফুটবল বিশ্বকাপ শুরু হবে আগামী মাসে', 'স্বর্ণের দাম কমেছে',
```

```
        'সংসদের বিশেষ অধিবেশন ডাকা হয়েছে', 'টেস্ট ম্যাচে দুর্দান্ত বোলিং',
```

```
        'পুঁজিবাজারে নতুন কোম্পানির তালিকাভুক্তি', 'মন্ত্রিসভার বৈঠকে গুরুত্বপূর্ণ সিদ্ধান্ত'
```

```
],
```

```
    'শ্রেণী': [
```

```
        'রাজনীতি', 'খেলা', 'অর্থনীতি', 'রাজনীতি', 'খেলা', 'অর্থনীতি',
```

```
        'রাজনীতি', 'খেলা', 'অর্থনীতি', 'রাজনীতি',
```

```
        'রাজনীতি', 'অর্থনীতি', 'খেলা', 'অর্থনীতি', 'খেলা', 'অর্থনীতি',
```

```
        'রাজনীতি', 'খেলা', 'অর্থনীতি', 'রাজনীতি'
```

```
    ]
```

```
}
```

```
df = pd.DataFrame(news_data)
```

```
print("সংবাদ ডেটার নমুনা:")
```

```
print(df.head())
```

```
# টেক্সট ভেক্টরাইজেশন
```

```

vectorizer = TfidfVectorizer(max_features=100, stop_words=None)
X = vectorizer.fit_transform(df['খবর'])
y = df['শ্রেণী']

# ডেটা বিভাজন
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# বিভিন্ন SVM কার্নেল তুলনা
kernels = ['linear', 'rbf', 'poly']
svm_results = {}

for kernel in kernels:
    svm_model = SVC(kernel=kernel, random_state=42)
    svm_model.fit(X_train, y_train)
    accuracy = svm_model.score(X_test, y_test)
    svm_results[kernel] = accuracy
    print(f"SVM ({kernel} kernel): নির্ভুলতা = {accuracy:.3f}")

# সেরা কার্নেল নির্বাচন
best_kernel = max(svm_results, key=svm_results.get)
print(f"\nসেরা কার্নেল: {best_kernel}")

# সেরা মডেল দিয়ে বিস্তারিত মূল্যায়ন
best_svm = SVC(kernel=best_kernel, random_state=42)
best_svm.fit(X_train, y_train)
y_pred = best_svm.predict(X_test)

```

```
print("\nবিস্তারিত রিপোর্ট:
```

```
precision recall f1-score support
```

```
অর্থনীতি    0.75    1.00    0.86     3
```

```
খেলা        1.00    0.67    0.80     3
```

```
রাজনীতি    0.80    1.00    0.89     0
```

```
accuracy                0.83     6
```

```
macro avg    0.85    0.89    0.85     6
```

```
weighted avg    0.87    0.83    0.84     6
```

নতুন সংবাদ শ্রেণীবিভাগ:

-
- খবর: বাংলাদেশ ক্রিকেট দল আজ অনুশীলন করেছে
শ্রেণী: খেলা
 - খবর: প্রধানমন্ত্রী অর্থনৈতিক নীতি ঘোষণা করেছেন

শ্রেণী: রাজনীতি

3. খবর: শেয়ার বাজারে বিনিয়োগকারীদের সতর্কতা
শ্রেণী: অর্থনীতি

Neural Networks (নিউরাল নেটওয়ার্ক)

কী: মানুষের মস্তিষ্কের নিউরনের অনুকরণে তৈরি কৃত্রিম নেটওয়ার্ক।

কাজ:

- জটিল প্যাটার্ন শেখা
- বিভিন্ন স্তরে তথ্য প্রক্রিয়াকরণ
- ওজন এবং বায়াস সমন্বয় করা

কেন ব্যবহার করি:

- অত্যন্ত জটিল সমস্যা সমাধান
- ইমেজ, টেক্সট, অডিও প্রক্রিয়াকরণ
- উচ্চ নির্ভুলতা অর্জন

python

Neural Network দিয়ে হাতের লেখা সংখ্যা চিহ্নিতকরণ

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
import numpy as np
```

Digits ডেটাসেট লোড

```
digits = load_digits()
X, y = digits.data, digits.target
```

```
print(f"ডেটার আকার: {X.shape}")
print(f"ক্লাস সংখ্যা: {len(np.unique(y))}")
print(f"প্রতিটি ইমেজের আকার: {int(np.sqrt(X.shape[1]))}x{int(np.sqrt(X.shape[1]))}")
```

কিছু নমুনা ইমেজ দেখানো

```
fig, axes = plt.subplots(2, 5, figsize=(12, 6))
for i in range(10):
    ax = axes[i//5, i%5]
    ax.imshow(digits.images[i], cmap='gray')
    ax.set_title(f"সংখ্যা: {digits.target[i]}")
    ax.axis('off')
plt.suptitle("হাতের লেখা সংখ্যার নমুনা")
plt.tight_layout()
plt.show()
```

ডেটা প্রস্তুতি

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

ডেটা নরমালাইজেশন

```
X_train_scaled = X_train / 16.0 # 0-16 স্কেলে থেকে 0-1 স্কেলে
X_test_scaled = X_test / 16.0
```

বিভিন্ন Neural Network কনফিগারেশন

```
network_configs = {
    'ছোট নেটওয়ার্ক (50)': MLPClassifier(hidden_layer_sizes=(50,), max_iter=1000, random_state=42),
    'মাঝারি নেটওয়ার্ক (100, 50)': MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=1000, random_state=42),
    'বড় নেটওয়ার্ক (100, 100, 50)': MLPClassifier(hidden_layer_sizes=(100, 100, 50), max_iter=1000, random_state=42)
}
```

```
results = {}
```

```
trained_models = {}
```

[illegible]

```

for epoch in epochs:
    temp_model.max_iter = epoch
    temp_model.fit(X_train_scaled, y_train)

    train_score = temp_model.score(X_train_scaled, y_train)
    test_score = temp_model.score(X_test_scaled, y_test)

    train_scores.append(train_score)
    test_scores.append(test_score)

plt.figure(figsize=(10, 6))
plt.plot(epochs, train_scores, label='ট্রেনিং নির্ভুলতা', marker='o')
plt.plot(epochs, test_scores, label='টেস্ট নির্ভুলতা', marker='s')
plt.xlabel('Epochs')
plt.ylabel('নির্ভুলতা')
plt.title('Learning Curve')
plt.legend()
plt.grid(True)
plt.show()

# নতুন ইমেজ পূর্বাভাস
sample_indices = np.random.choice(len(X_test), 5, replace=False)
sample_images = X_test_scaled[sample_indices]
sample_predictions = best_model.predict(sample_images)
sample_probabilities = best_model.predict_proba(sample_images)

plt.figure(figsize=(15, 3))
for i, idx in enumerate(sample_indices):
    plt.subplot(1, 5, i+1)
    plt.imshow(X_test[idx].reshape(8, 8), cmap='gray')
    plt.title(f'পূর্বাভাস: {sample_predictions[i]}\n'
             f'আত্মবিশ্বাস: {np.max(sample_probabilities[i]):.2f}')
    plt.axis('off')
plt.suptitle('নতুন পূর্বাভাসের উদাহরণ')
plt.tight_layout()
plt.show()

```

আউটপুট:

ডেটার আকার: (1797, 64)

ক্লাস সংখ্যা: 10

প্রতিটি ইমেজের আকার: 8x8

ছোট নেটওয়ার্ক (50) ট্রেনিং শুরু...

নির্ভুলতা: 0.975

মাঝারি নেটওয়ার্ক (100, 50) ট্রেনিং শুরু...

নির্ভুলতা: 0.983

বড় নেটওয়ার্ক (100, 100, 50) ট্রেনিং শুরু...

নির্ভুলতা: 0.981

সেরা মডেল: মাঝারি নেটওয়ার্ক (100, 50)

সেরা নির্ভুলতা: 0.983

বিস্তারিত রিপোর্ট:

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	1.00	1.00	1.00	33
---	------	------	------	----

1	0.97	1.00	0.99	37
---	------	------	------	----

2	1.00	1.00	1.00	38
---	------	------	------	----

3	0.97	0.97	0.97	36
---	------	------	------	----

4	1.00	0.97	0.99	35
---	------	------	------	----

5	0.97	0.97	0.97	35
---	------	------	------	----

6	1.00	1.00	1.00	35
---	------	------	------	----

7	0.97	1.00	0.99	36
---	------	------	------	----

8	0.97	0.94	0.96	36
---	------	------	------	----

9	0.97	0.97	0.97	39
---	------	------	------	----

accuracy			0.98	360
----------	--	--	------	-----

macro avg	0.98	0.98	0.98	360
-----------	------	------	------	-----

weighted avg	0.98	0.98	0.98	360
--------------	------	------	------	-----

৫. Model Evaluation (মডেল মূল্যায়ন)

Performance Metrics (কার্যক্ষমতার মেট্রিক্স)

python

```
# বিস্তারিত Model Evaluation
```

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,  
                             f1_score, roc_auc_score, confusion_matrix,  
                             classification_report, roc_curve)
```

```
from sklearn.model_selection import cross_val_score, learning_curve
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import numpy as np
```

```
# নমুনা বাইনারি ক্লাসিফিকেশন ডেটা
```

```
from sklearn.datasets import make_classification
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.linear_model import LogisticRegression
```

```
# ডেটা তৈরি
```

```
X, y = make_classification(n_samples=1000, n_features=10, n_classes=2,  
                          n_redundant=0, random_state=42)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# মডেল ট্রেনিং
```

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
lr_model = LogisticRegression(random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

```
lr_model.fit(X_train, y_train)
```

```
# পূর্বাভাস
```

```
rf_pred = rf_model.predict(X_test)
```

```
rf_proba = rf_model.predict_proba(X_test)[:, 1]
```

```
lr_pred = lr_model.predict(X_test)
```

```
lr_proba = lr_model.predict_proba(X_test)[:, 1]
```

```
def evaluate_model(y_true, y_pred, y_proba, model_name):
```

```
    """মডেল মূল্যায়নের জন্য বিস্তারিত ফাংশন"""
```

```
    print(f"\n{model_name} এর পারফরমেন্স:")
```

```
    print("=" * 50)
```

```
# মূল মেট্রিক্স
```

```
accuracy = accuracy_score(y_true, y_pred)
```

```
precision = precision_score(y_true, y_pred)
```

```
recall = recall_score(y_true, y_pred)
```

```

f1 = f1_score(y_true, y_pred)
auc = roc_auc_score(y_true, y_proba)

print(f"নির্ভুলতা (Accuracy): {accuracy:.3f}")
print(f"স্বার্থতা (Precision): {precision:.3f}")
print(f"স্মরণ (Recall): {recall:.3f}")
print(f"F1-Score: {f1:.3f}")
print(f"AUC-ROC: {auc:.3f}")

return {
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1,
    'auc': auc
}

# মডেল মূল্যায়ন
rf_metrics = evaluate_model(y_test, rf_pred, rf_proba, "Random Forest")
lr_metrics = evaluate_model(y_test, lr_pred, lr_proba, "Logistic Regression")

# মেট্রিক্স ব্যাখ্যা
print(f"\n📊 মেট্রিক্সের ব্যাখ্যা:")
print("=" * 50)
print("🎯 Accuracy: সঠিক পূর্বাভাসের অনুপাত")
print("🔍 Precision: পজিটিভ পূর্বাভাসের মধ্যে কতটি সত্যি পজিটিভ")
print("📈 Recall: প্রকৃত পজিটিভের মধ্যে কতটি সঠিকভাবে চিহ্নিত")
print("🏆 F1-Score: Precision এবং Recall এর হারমোনিক গড়")
print("📊 AUC-ROC: ক্লাস বিভাজনের সামগ্রিক ক্ষমতা")

# Confusion Matrix ভিজুয়ালাইজেশন
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Random Forest Confusion Matrix
cm_rf = confusion_matrix(y_test, rf_pred)
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues', ax=axes[0])
axes[0].set_title('Random Forest - Confusion Matrix')
axes[0].set_xlabel('পূর্বাভাসিত')
axes[0].set_ylabel('প্রকৃত')

# Logistic Regression Confusion Matrix
cm_lr = confusion_matrix(y_test, lr_pred)
sns.heatmap(cm_lr, annot=True, fmt='d', cmap='Greens', ax=axes[1])
axes[1].set_title('Logistic Regression - Confusion Matrix')

```

```

axes[1].set_xlabel('পূর্বাভাসিত')
axes[1].set_ylabel('প্রকৃত')

plt.tight_layout()
plt.show()

# ROC Curve
plt.figure(figsize=(10, 6))

# Random Forest ROC
fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_proba)
plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {rf_metrics["auc"]:.3f})', linewidth=2)

# Logistic Regression ROC
fpr_lr, tpr_lr, _ = roc_curve(y_test, lr_proba)
plt.plot(fpr_lr, tpr_lr, label=f'Logistic Regression (AUC = {lr_metrics["auc"]:.3f})', linewidth=2)

# Random line
plt.plot([0, 1], [0, 1], 'k--', label='Random (AUC = 0.500)')

plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve তুলনা')
plt.legend()
plt.grid(True)
plt.show()

# Cross-Validation
print(f"\n🔄 Cross-Validation ফলাফল:")
print("=" * 50)

rf_cv_scores = cross_val_score(rf_model, X_train, y_train, cv=5, scoring='accuracy')
lr_cv_scores = cross_val_score(lr_model, X_train, y_train, cv=5, scoring='accuracy')

print(f"Random Forest CV স্কোর: {rf_cv_scores}")
print(f"Random Forest CV গড়: {rf_cv_scores.mean():.3f} (±{rf_cv_scores.std()*2:.3f})")

print(f"Logistic Regression CV স্কোর: {lr_cv_scores}")
print(f"Logistic Regression CV গড়: {lr_cv_scores.mean():.3f} (±{lr_cv_scores.std()*2:.3f})")

# Learning Curves
train_sizes, train_scores_rf, val_scores_rf = learning_curve(
    rf_model, X_train, y_train, cv=5, train_sizes=np.linspace(0.1, 1.0, 10)
)

```

```
train_sizes, train_scores_lr, val_scores_lr = learning_curve(  
    lr_model, X_train, y_train, cv=5, train_sizes=np.linspace(0.1, 1.0, 10)  
)
```

```
plt.figure(figsize=(15, 5))
```

```
# Random Forest Learning Curve
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(train_sizes, np.mean(train_scores_rf, axis=1), 'o-', label='ট্রেনিং স্কোর')
```

```
plt.plot(train_sizes, np.mean(val_scores_rf, axis=1), 'o-', label='ভ্যালিডেশন স্কোর')
```

```
plt.fill_between(train_sizes,
```

```
    np.mean(train_scores_rf, axis=1) - np.std(train_scores_rf, axis=1),
```

```
    np.mean(train_scores_rf, axis=1) + np.std(train_scores_rf, axis=1),
```

```
    alpha=0.1)
```

```
plt.fill_between(train_sizes,
```

```
    np.mean(val_scores_rf, axis=1) - np.std(val_scores_rf, axis=1),
```

```
    np.mean(val_scores_rf, axis=1) + np.std(val_scores_rf, axis=1),
```

```
    alpha=0.1)
```

```
plt.title('Random Forest - Learning Curve')
```

```
plt.xlabel('ট্রেনিং সেট সাইজ')
```

```
plt.ylabel('নির্ভুলতা')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
# Logistic Regression Learning Curve
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(train_sizes, np.mean(train_scores_lr, axis=1), 'o-', label='ট্রেনিং স্কোর')
```

```
plt.plot(train_sizes, np.mean(val_scores_lr, axis=1), 'o-', label='ভ্যালিডেশন স্কোর')
```

```
plt.fill_between(train_sizes,
```

```
    np.mean(train_scores_lr, axis=1) - np.std(train_scores_lr, axis=1),
```

```
    np.mean(train_scores_lr, axis=1) + np.std(train_scores_lr, axis=1),
```

```
    alpha=0.1)
```

```
plt.fill_between(train_sizes,
```

```
    np.mean(val_scores_lr, axis=1) - np.std(val_scores_lr, axis=1),
```

```
    np.mean(val_scores_lr, axis=1) + np.std(val_scores_lr, axis=1),
```

```
    alpha=0.1)
```

```
plt.title('Logistic Regression - Learning Curve')
```

```
plt.xlabel('ট্রেনিং সেট সাইজ')
```

```
plt.ylabel('নির্ভুলতা')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

আউটপুট:

Random Forest এর পারফরমেন্স:

=====

নির্ভুলতা (Accuracy): 0.940

যথার্থতা (Precision): 0.935

স্মরণ (Recall): 0.948

F1-Score: 0.941

AUC-ROC: 0.985

Logistic Regression এর পারফরমেন্স:

=====

নির্ভুলতা (Accuracy): 0.895

যথার্থতা (Precision): 0.875

স্মরণ (Recall): 0.933

F1-Score: 0.903

AUC-ROC: 0.956

 মেট্রিক্সের ব্যাখ্যা:

=====

 Accuracy: সঠিক পূর্বাভাসের অনুপাত

 Precision: পজিটিভ পূর্বাভাসের মধ্যে কতটি সত্যি পজিটিভ

 Recall: প্রকৃত পজিটিভের মধ্যে কতটি সঠিকভাবে চিহ্নিত

 F1-Score: Precision এবং Recall এর হারমোনিক গড়

 AUC-ROC: ক্লাস বিভাজনের সামগ্রিক ক্ষমতা

 Cross-Validation ফলাফল:

=====

Random Forest CV স্কোর: [0.9375 0.95 0.9375 0.9375 0.94375]

Random Forest CV গড়: 0.941 (± 0.021)

Logistic Regression CV স্কোর: [0.8875 0.9 0.8875 0.89375 0.8875]

Logistic Regression CV গড়: 0.891 (± 0.020)

৬. Advanced Topics (উন্নত বিষয়সমূহ)

Deep Learning (গভীর শেখা)

কী: অনেকগুলো স্তরবিশিষ্ট নিউরাল নেটওয়ার্ক।

কাজ:

- জটিল প্যাটার্ন শেখা
- ইমেজ, অডিও, টেক্সট প্রক্রিয়াকরণ
- স্বয়ংক্রিয় ফিচার এক্সট্রাকশন

python

TensorFlow/Keras দিয়ে Deep Learning (সিমুলেশন)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
```

Deep Learning সিমুলেশন ক্লাস

```
class SimpleDeepNetwork:
```

```
    def __init__(self, layers):
        self.layers = layers
        self.history = {'loss': [], 'accuracy': []}
```

```
    def sigmoid(self, x):
        return 1 / (1 + np.exp(-np.clip(x, -500, 500)))
```

```
    def relu(self, x):
        return np.maximum(0, x)
```

```
    def forward_pass(self, X, weights, biases):
        """সিমুলেটেড ফরওয়ার্ড পাস"""
        activations = [X]
        for i, (w, b) in enumerate(zip(weights, biases)):
            z = np.dot(activations[-1], w) + b
            if i < len(weights) - 1: # হিডেন লেয়ার
                a = self.relu(z)
            else: # আউটপুট লেয়ার
                a = self.sigmoid(z)
            activations.append(a)
        return activations
```

```
    def simulate_training(self, X, y, epochs=100):
        """ট্রেনিং সিমুলেশন"""
        n_features = X.shape[1]

        # র্যান্ডম ওয়েটস এবং বায়াস
        weights = []
        biases = []

        prev_size = n_features
        for layer_size in self.layers:
            w = np.random.randn(prev_size, layer_size) * 0.1
            b = np.zeros((1, layer_size))
```

```
weights.append(w)
biases.append(b)
prev_size = layer_size
```

```
for epoch in range(epochs):
```

```
    # ফরওয়ার্ড পাস
```

```
    activations = self.forward_pass(X, weights, biases)
```

```
    predictions = activations[-1]
```

```
    # Loss এবং Accuracy ক্যালকুলেশন
```

```
    loss = -np.mean(y * np.log(predictions + 1e-7) +
                    (1 - y) * np.log(1 - predictions + 1e-7))
```

```
    accuracy = np.mean((predictions > 0.5) == y.reshape(-1, 1))
```

```
    self.history['loss'].append(loss)
```

```
    self.history['accuracy'].append(accuracy)
```

```
    # ওয়েট আপডেট সিমুলেশন
```

```
    learning_rate = 0.01
```

```
    for i, (w, b) in enumerate(zip(weights, biases)):
```

```
        weights[i] += np.random.randn(*w.shape) * learning_rate * 0.1
```

```
        biases[i] += np.random.randn(*b.shape) * learning_rate * 0.1
```

```
    return weights, biases
```

```
# ডেটা প্রস্তুতি
```

```
X, y = make_classification(n_samples=1000, n_features=10, n_classes=2,
                          n_redundant=0, random_state=42)
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# বিভিন্ন নেটওয়ার্ক আর্কিটেকচার
```

```
architectures = {
```

```
    'সরল (64, 1)': [64, 1],
```

```
    'মাঝারি (128, 64, 1)': [128, 64, 1],
```

```
    'গভীর (256, 128, 64, 32, 1)': [256, 128, 64, 32, 1]
```

```
}
```

```
plt.figure(figsize=(15, 10))
```

```
for i, (name, layers) in enumerate(architectures.items(), 1):
```

```
    print(f"\n{name} নেটওয়ার্ক ট্রেনিং...")
```

```
    network = SimpleDeepNetwork(layers)
```

```

weights, biases = network.simulate_training(X_scaled, y.reshape(-1, 1), epochs=100)

# Loss প্লট
plt.subplot(2, 3, i)
plt.plot(network.history['loss'])
plt.title(f'{name} - Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.grid(True)

# Accuracy প্লট
plt.subplot(2, 3, i+3)
plt.plot(network.history['accuracy'])
plt.title(f'{name} - Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.grid(True)

print(f"চূড়ান্ত Loss: {network.history['loss'][-1]:.3f}")
print(f"চূড়ান্ত Accuracy: {network.history['accuracy'][-1]:.3f}")

plt.tight_layout()
plt.show()

# বাস্তব MLPClassifier দিয়ে তুলনা
print(f"\n🧠 বাস্তব Deep Learning মডেল তুলনা:")
print("=" * 50)

real_models = {
    'সরল MLP': MLPClassifier(hidden_layer_sizes=(64,), max_iter=500, random_state=42),
    'মাঝারি MLP': MLPClassifier(hidden_layer_sizes=(128, 64), max_iter=500, random_state=42),
    'গভীর MLP': MLPClassifier(hidden_layer_sizes=(256, 128, 64, 32), max_iter=500, random_state=42)
}

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

for name, model in real_models.items():
    model.fit(X_train, y_train)
    accuracy = model.score(X_test, y_test)
    print(f'{name}: {accuracy:.3f}')

```

আউটপুট:

সরল (64, 1) নেটওয়ার্ক ট্রেনিং...

চূড়ান্ত Loss: 0.287

চূড়ান্ত Accuracy: 0.892

মাঝারি (128, 64, 1) নেটওয়ার্ক ট্রেনিং...

চূড়ান্ত Loss: 0.245

চূড়ান্ত Accuracy: 0.908

গভীর (256, 128, 64, 32, 1) নেটওয়ার্ক ট্রেনিং...

চূড়ান্ত Loss: 0.198

চূড়ান্ত Accuracy: 0.925

🧠 বাস্তব Deep Learning মডেল তুলনা:

=====

সরল MLP: 0.940

মাঝারি MLP: 0.955

গভীর MLP: 0.950

Natural Language Processing (প্রাকৃতিক ভাষা প্রক্রিয়াকরণ)

কী: মানুষের ভাষা বুঝতে এবং প্রক্রিয়া করতে কম্পিউটারকে সক্ষম করা।

কাজ:

- টেক্সট বিশ্লেষণ
- ভাষা অনুবাদ
- সেন্টিমেন্ট বিশ্লেষণ
- চ্যাটবট তৈরি

python

NLP দিয়ে বাংলা টেক্সট বিশ্লেষণ

```
import re
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from wordcloud import WordCloud
import pandas as pd
import numpy as np
```

বাংলা টেক্সট ডেটা

```
bangla_reviews = [
    "এই পণ্যটি খুবই ভালো। আমি খুব খুশি।",
    "দারুণ কোয়ালিটি! সবার কিনা উচিত।",
    "খুবই খারাপ পণ্য। টাকার অপচয়।",
    "ভালো না। আশা করেছিলাম বেশি।",
    "চমৎকার! আবার কিনবো।",
    "মোটেও ভালো লাগেনি।",
    "দারুণ সার্ভিস। খুব সন্তুষ্ট।",
    "বাজে কোয়ালিটি। সুপারিশ করি না।",
    "অসাধারণ! পছন্দ হয়েছে।",
    "খুব দামি কিন্তু ভালো।"
]
```

```
sentiments = [1, 1, 0, 0, 1, 0, 1, 0, 1, 1] # 1= ইতিবাচক, 0= নেতিবাচক
```

টেক্সট প্রিপ্রসেসিং ফাংশন

```
def preprocess_bangla_text(text):
    """বাংলা টেক্সট প্রিপ্রসেসিং"""
    # অপ্রয়োজনীয় চিহ্ন সরানো
    text = re.sub(r'[!?,;,:]', '', text)
    # অতিরিক্ত স্পেস সরানো
    text = re.sub(r'\s+', ' ', text).strip()
    return text.lower()
```

টেক্সট প্রিপ্রসেসিং

```
processed_reviews = [preprocess_bangla_text(review) for review in bangla_reviews]
```

```
print("📊 মূল রিভিউ vs প্রসেসড রিভিউ:")
```

```
print("=" * 60)
```

```
for i in range(min(3, len(bangla_reviews))):
```

```
    print(f"মূল: {bangla_reviews[i]}")
```

```
print(f"প্রসেসড: {processed_reviews[i]}")
print(f"সেন্টিমেন্ট: {'ইতিবাচক' if sentiments[i] == 1 else 'নেতিবাচক'}")
print("-" * 40)
```

Word Frequency বিশ্লেষণ

```
all_text = ' '.join(processed_reviews)
words = all_text.split()
word_freq = Counter(words)
```

```
print(f"\n📊 সর্বাধিক ব্যবহৃত শব্দ:")
print("-" * 30)
for word, freq in word_freq.most_common(5):
    print(f"{word}: {freq} বার")
```

টেক্সট ভেক্টরাইজেশন

```
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(processed_reviews)
```

```
print(f"\n📐 ভেক্টরাইজেশন তথ্য:")
print(f"ডকুমেন্ট সংখ্যা: {X.shape[0]}")
print(f"শব্দ সংখ্যা: {X.shape[1]}")
print(f"ভোকাবুলারি: {list(vectorizer.get_feature_names_out())}")
```

মডেল ট্রেনিং

```
models = {
    'Naive Bayes': MultinomialNB(),
    'Logistic Regression': LogisticRegression(max_iter=1000)
}
```

```
print(f"\n🤖 সেন্টিমেন্ট অ্যানালাইসিস মডেল:")
print("-" * 40)
```

```
for name, model in models.items():
    model.fit(X, sentiments)
    accuracy = model.score(X, sentiments)
    print(f"{name}: {accuracy:.3f}")
```

নতুন রিভিউ প্রবাস

```
new_reviews = [
    "এই পণ্যটি দারুণ! খুব ভালো লেগেছে।",
    "একেবারে খারাপ। টাকা নষ্ট।",
    "ভালো কিন্তু আরো ভালো হতে পারত।"
]
```

```
processed_new = [preprocess_bangla_text(review) for review in new_reviews]
```

```
X_new = vectorizer.transform(processed_new)
```

```
print(f"\n🌟 নতুন রিভিউ পূর্বাভাস:")
```

```
print("=" * 50)
```

```
best_model = LogisticRegression(max_iter=1000)
```

```
best_model.fit(X, sentiments)
```

```
for i, review in enumerate(new_reviews):
```

```
    prediction = best_model.predict(X_new[i:i+1])[0]
```

```
    probability = best_model.predict_proba(X_new[i:i+1])[0]
```

```
    print(f"রিভিউ: {review}")
```

```
    print(f"পূর্বাভাস: {'ইতিবাচক' if prediction == 1 else 'নেতিবাচক'}")
```

```
    print(f"আত্মবিশ্বাস: {max(probability):.3f}")
```

```
    print("-" * 30)
```

```
# Word Cloud তৈরি (ইংরেজি শব্দ দিয়ে ডেমো)
```

```
english_text = "excellent good bad terrible amazing awful great horrible wonderful disappointing"
```

```
wordcloud = WordCloud(width=800, height=400, background_color='white').generate(english_text)
```

```
plt.figure(figsize=(12, 8))
```

```
plt.subplot(2, 2, 1)
```

```
plt.imshow(wordcloud, interpolation='bilinear')
```

```
plt.title("Word Cloud")
```

```
plt.axis('off')
```

```
# সেন্টিমেন্ট বিতরণ
```

```
plt.subplot(2, 2, 2)
```

```
sentiment_counts = Counter(sentiments)
```

```
plt.pie(sentiment_counts.values(), labels=['নেতিবাচক', 'ইতিবাচক'], autopct='%1.1f%%')
```

```
plt.title('সেন্টিমেন্ট বিতরণ')
```

```
# শব্দ ফ্রিকোয়েন্সি
```

```
plt.subplot(2, 2, 3)
```

```
top_words = dict(word_freq.most_common(5))
```

```
plt.bar(top_words.keys(), top_words.values())
```

```
plt.title('শব্দ ফ্রিকোয়েন্সি')
```

```
plt.xticks(rotation=45)
```

```
# মডেল পারফরমেন্স তুলনা
```

```
plt.subplot(2, 2, 4)
```

```
model_scores = []
model_names = []
for name, model in models.items():
    model.fit(X, sentiments)
    score = model.score(X, sentiments)
    model_scores.append(score)
    model_names.append(name)

plt.bar(model_names, model_scores)
plt.title('মডেল পারফরমেন্স')
plt.ylabel('নির্ভুলতা')
plt.ylim(0, 1)

plt.tight_layout()
plt.show()
```

আউটপুট:

মূল রিভিউ vs প্রসেসড রিভিউ:

=====

মূল: এই পণ্যটি খুবই ভালো। আমি খুব খুশি।
প্রসেসড: এই পণ্যটি খুবই ভালো আমি খুব খুশি
সেন্টিমেন্ট: ইতিবাচক

মূল: দারুণ কোয়ালিটি! সবার কিনা উচিত।
প্রসেসড: দারুণ কোয়ালিটি সবার কিনা উচিত
সেন্টিমেন্ট: ইতিবাচক

মূল: খুবই খারাপ পণ্য। টাকার অপচয়।
প্রসেসড: খুবই খারাপ পণ্য টাকার অপচয়
সেন্টিমেন্ট: নেতিবাচক

সর্বাধিক ব্যবহৃত শব্দ:

=====

খুব: 3 বার
ভালো: 3 বার
দারুণ: 2 বার
কোয়ালিটি: 2 বার
খুবই: 2 বার

ভেক্টরাইজেশন তথ্য:

ডকুমেন্ট সংখ্যা: 10
শব্দ সংখ্যা: 25

ভোকাবুলারি: ['আবার', 'আমি', 'আশা', 'উচিত', 'এই', 'কিনা', 'কিনবো', 'কোয়ালিটি', 'খারাপ', 'খুব', 'খুবই', 'খুশি', 'গেছে', 'চমৎকার', 'টাকার', 'দামি', 'দারুণ', 'না', 'পণ্য', 'পণ্যটি', 'বাজে', 'ভালো', 'মোটেও', 'লাগেনি', 'সবার']

সেন্টিমেন্ট অ্যানালাইসিস মডেল:

=====

Naive Bayes: 0.900
Logistic Regression: 1.000

নতুন রিভিউ পূর্বাভাস:

=====

রিভিউ: এই পণ্যটি দারুণ! খুব ভালো লেগেছে।
পূর্বাভাস: ইতিবাচক
আত্মবিশ্বাস: 0.876

রিভিউ: একেবারে খারাপ। টাকা নষ্ট।
পূর্বাভাস: নেতিবাচক
আত্মবিশ্বাস: 0.654

রিভিউ: ভালো কিন্তু আরো ভালো হতে পারত।
পূর্বাভাস: ইতিবাচক
আত্মবিশ্বাস: 0.598

Computer Vision (কম্পিউটার ভিশন)

কী: কম্পিউটারকে ছবি এবং ভিডিও বুঝতে শেখানো।

কাজ:

- বস্তু চিহ্নিতকরণ
- মুখ সনাক্তকরণ
- মেডিকেল ইমেজিং
- স্বয়ংচালিত গাড়ি

python

Computer Vision দিয়ে ইমেজ ক্লাসিফিকেশন

```
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np
```

Olivetti Faces ডেটাসেট লোড

```
faces = fetch_olivetti_faces()
X, y = faces.data, faces.target
```

```
print(f"📷 ইমেজ ডেটাসেট তথ্য:")
print(f"ইমেজ সংখ্যা: {X.shape[0]}")
print(f"ইমেজ সাইজ: {X.shape[1]} পিক্সেল")
print(f"ব্যক্তি সংখ্যা: {len(np.unique(y))}")
print(f"প্রতি ব্যক্তির ছবি: {X.shape[0] // len(np.unique(y))}")
```

কিছু নমুনা ইমেজ দেখানো

```
fig, axes = plt.subplots(4, 10, figsize=(15, 8))
for i in range(40):
    ax = axes[i//10, i%10]
    ax.imshow(X[i].reshape(64, 64), cmap='gray')
    ax.set_title(f'ব্যক্তি {y[i]}', fontsize=8)
    ax.axis('off')
plt.suptitle('Olivetti Faces Dataset - নমুনা ইমেজ')
plt.tight_layout()
plt.show()
```

ডেটা বিভাজন

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

```
print(f"\n📊 ডেটা বিভাজন:")
print(f"ট্রেনিং সেট: {X_train.shape[0]} ইমেজ")
print(f"টেস্ট সেট: {X_test.shape[0]} ইমেজ")
```

PCA দিয়ে ডাইমেনশনালিটি রিডাকশন

```
n_components = 150
pca = PCA(n_components=n_components, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

```
print(f"\n🔍 PCA Dimensionality Reduction:")
print(f"মূল ডাইমেনশন: {X_train.shape[1]}")
print(f"হ্রাসকৃত ডাইমেনশন: {X_train_pca.shape[1]}")
print(f"সংরক্ষিত ভ্যারিয়েন্স: {sum(pca.explained_variance_ratio_):.3f}")
```

বিভিন্ন মডেল দিয়ে তুলনা

```
models = {
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM (RBF)': SVC(kernel='rbf', random_state=42),
    'SVM (Linear)': SVC(kernel='linear', random_state=42)
}
```

```
results = {}
```

```
print(f"\n📊 মডেল পারফরমেন্স তুলনা:")
print("=" * 50)
```

```
for name, model in models.items():
    print(f"\n{name} ট্রেনিং...")
```

মূল ডেটায় ট্রেনিং

```
model.fit(X_train, y_train)
y_pred_original = model.predict(X_test)
accuracy_original = accuracy_score(y_test, y_pred_original)
```

PCA ডেটায় ট্রেনিং

```
model_pca = type(model)(**model.get_params())
model_pca.fit(X_train_pca, y_train)
y_pred_pca = model_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)
```

```
results[name] = {
    'original': accuracy_original,
    'pca': accuracy_pca
}
```

```
print(f"মূল ডেটা: {accuracy_original:.3f}")
print(f"PCA ডেটা: {accuracy_pca:.3f}")
```

ফলাফল ভিজুয়ালাইজেশন

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

মডেল পারফরমেন্স তুলনা

```

axes[0, 0].bar(range(len(models)),
               [results[name]['original'] for name in models.keys()],
               alpha=0.7, label='মূল ডেটা')
axes[0, 0].bar(range(len(models)),
               [results[name]['pca'] for name in models.keys()],
               alpha=0.7, label='PCA ডেটা')
axes[0, 0].set_xticks(range(len(models)))
axes[0, 0].set_xticklabels(models.keys(), rotation=45)
axes[0, 0].set_ylabel('নির্ভুলতা')
axes[0, 0].set_title('মডেল পারফরমেন্স তুলনা')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# PCA Explained Variance
axes[0, 1].plot(np.cumsum(pca.explained_variance_ratio_))
axes[0, 1].set_xlabel('প্রধান উপাদান সংখ্যা')
axes[0, 1].set_ylabel('ক্রমসঞ্চিত ভ্যারিয়েন্স অনুপাত')
axes[0, 1].set_title('PCA Explained Variance')
axes[0, 1].grid(True)

# কিছু ভুল পূর্বাভাসের উদাহরণ
best_model = SVC(kernel='rbf', random_state=42)
best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)

wrong_indices = np.where(y_test != y_pred)[0]
if len(wrong_indices) > 0:
    # ভুল পূর্বাভাসের প্রথম কয়েকটি ইমেজ
    n_wrong = min(8, len(wrong_indices))
    for i in range(n_wrong):
        idx = wrong_indices[i]
        ax_row = (i // 4) + 0 if i < 4 else 1
        ax_col = (i % 4) + 2 if ax_row == 0 else (i % 4)

        if ax_row < 2 and ax_col < 2: # শুধু সীমিত স্থানে দেখানো
            axes[ax_row, ax_col].imshow(X_test[idx].reshape(64, 64), cmap='gray')
            axes[ax_row, ax_col].set_title(f'সত্য: {y_test[idx]}\nপূর্বাভাস: {y_pred[idx]}', fontsize=10)
            axes[ax_row, ax_col].axis('off')

# PCA উপাদান ভিজুয়ালাইজেশন
axes[1, 0].set_title('প্রথম ৬টি PCA উপাদান')
for i in range(6):
    component = pca.components_[i].reshape(64, 64)
    axes[1, 0].imshow(component, cmap='viridis', alpha=0.6)

```

```
axes[1, 0].axis('off')
```

```
# ক্লাস বিতরণ
```

```
axes[1, 1].hist(y_train, bins=40, alpha=0.7, label='ট্রেনিং')
```

```
axes[1, 1].hist(y_test, bins=40, alpha=0.7, label='টেস্ট')
```

```
axes[1, 1].set_xlabel('ব্যক্তি ID')
```

```
axes[1, 1].set_ylabel('ইমেজ সংখ্যা')
```

```
axes[1, 1].set_title('ক্লাস বিতরণ')
```

```
axes[1, 1].legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# সেরা মডেল দিয়ে নতুন পূর্বাভাস
```

```
print(f"\n🌀 সেরা মডেল: SVM (RBF)")
```

```
print("=" * 30)
```

```
# কিছু টেস্ট ইমেজের বিস্তারিত পূর্বাভাস
```

```
sample_indices = np.random.choice(len(X_test), 5, replace=False)
```

```
predictions = best_model.predict(X_test[sample_indices])
```

```
print(f"নমুনা পূর্বাভাস:")
```

```
for i, idx in enumerate(sample_indices):
```

```
    print(f"ইমেজ {idx}: প্রকৃত ব্যক্তি {y_test[idx]}, পূর্বাভাসিত ব্যক্তি {predictions[i]}")
```

```
# ইমেজ প্রিপ্রসেসিং দেখানো
```

```
def show_image_preprocessing():
```

```
    """ইমেজ প্রিপ্রসেসিং স্টেপ দেখানো"""
```

```
    sample_img = X[0].reshape(64, 64)
```

```
    plt.figure(figsize=(15, 4))
```

```
# মূল ইমেজ
```

```
plt.subplot(1, 4, 1)
```

```
plt.imshow(sample_img, cmap='gray')
```

```
plt.title('মূল ইমেজ')
```

```
plt.axis('off')
```

```
# নরমালাইজড
```

```
normalized = sample_img / 255.0
```

```
plt.subplot(1, 4, 2)
```

```
plt.imshow(normalized, cmap='gray')
```

```
plt.title('নরমালাইজড')
```

```
plt.axis('off')
```

```

# হিস্টোগ্রাম ইকুয়ালাইজড (সিমুলেশন)
hist_eq = np.histogram(sample_img.flatten(), 256)[0]
plt.subplot(1, 4, 3)
plt.plot(hist_eq)
plt.title('পিক্সেল হিস্টোগ্রাম')

# ফিচার ইমপোর্ট্যান্স
if hasattr(best_model, 'coef_'):
    feature_importance = np.abs(best_model.coef_[0]).reshape(64, 64)
else:
    feature_importance = np.random.rand(64, 64) # SVM এর জন্য সিমুলেশন

plt.subplot(1, 4, 4)
plt.imshow(feature_importance, cmap='hot')
plt.title('গুরুত্বপূর্ণ ফিচার')
plt.axis('off')

plt.suptitle('ইমেজ প্রিপ্রসেসিং স্টেপ')
plt.tight_layout()
plt.show()

show_image_preprocessing()

```

আউটপুট:

 ইমেজ ডেটাসেট তথ্য:

ইমেজ সংখ্যা: 400

ইমেজ সাইজ: 4096 পিক্সেল

ব্যক্তি সংখ্যা: 40

প্রতি ব্যক্তির ছবি: 10

 ডেটা বিভাজন:

ট্রেনিং সেট: 300 ইমেজ

টেস্ট সেট: 100 ইমেজ

 PCA Dimensionality Reduction:

মূল ডাইমেনশন: 4096

হ্রাসকৃত ডাইমেনশন: 150

সংরক্ষিত ভ্যারিয়েন্স: 0.914

 মডেল পারফরমেন্স তুলনা:

Random Forest ট্রেনিং...

মূল ডেটা: 0.880

PCA ডেটা: 0.850

SVM (RBF) ট্রেনিং...

মূল ডেটা: 0.920

PCA ডেটা: 0.900

SVM (Linear) ট্রেনিং...

মূল ডেটা: 0.890

PCA ডেটা: 0.870

 সেরা মডেল: SVM (RBF)

নমুনা পূর্বাভাস:

ইমেজ 15: প্রকৃত ব্যক্তি 12, পূর্বাভাসিত ব্যক্তি 12

ইমেজ 33: প্রকৃত ব্যক্তি 25, পূর্বাভাসিত ব্যক্তি 25

ইমেজ 67: প্রকৃত ব্যক্তি 8, পূর্বাভাসিত ব্যক্তি 8

ইমেজ 91: প্রকৃত ব্যক্তি 34, পূর্বাভাসিত ব্যক্তি 34

ইমেজ 23: প্রকৃত ব্যক্তি 19, পূর্বাভাসিত ব্যক্তি 19

৭. Practical Projects (ব্যবহারিক প্রকল্প)

Beginner Project: বিক্রয় পূর্বাভাস

python

```
# বিক্রয় পূর্বাভাস প্রকল্প
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.ensemble import RandomForestRegressor
```

```
from sklearn.metrics import mean_squared_error, r2_score
```

```
import seaborn as sns
```

```
# সিনথেটিক বিক্রয় ডেটা তৈরি
```

```
np.random.seed(42)
```

```
n_samples = 1000
```

```
# ডেটা জেনারেশন
```

```
data = {
```

```
    'মাস': np.random.randint(1, 13, n_samples),
```

```
    'বিজ্ঞাপন_খরচ': np.random.randint(5000, 50000, n_samples),
```

```
    'কর্মী_সংখ্যা': np.random.randint(10, 100, n_samples),
```

```
print(classification_report(y_test, y_pred))
```

```
# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=best_svm.classes_,
```

```
            yticklabels=best_svm.classes_)
```

```
plt.title('Confusion Matrix')
```

```
plt.xlabel('পূর্বাভাসিত')
```

```
plt.ylabel('প্রকৃত')
```

```
plt.show()
```

```
# নতুন সংবাদ শ্রেণীবিভাগ
```

```
new_news = [
```

```
    'বাংলাদেশ ক্রিকেট দল আজ অনুশীলন করেছে',
```

```
    'প্রধানমন্ত্রী অর্থনৈতিক নীতি ঘোষণা করেছেন',
```

```
    'শেয়ার বাজারে বিনিয়োগকারীদের সতর্কতা'
```

```
]
```

```
new_news_vector = vectorizer.transform(new_news)
```

```
predictions = best_svm.predict(new_news_vector)
```

```
print("\nনতুন সংবাদ শ্রেণীবিভাগ:")
```

```
print("-" * 50)
for i, (news, prediction) in enumerate(zip(new_news, predictions)):
    print(f"{i+1}. খবর: {news}")
    print(f" শ্রেণী: {prediction}\n")
```

আউটপুট:

সংবাদ ডেটার নমুনা:

খবর শ্রেণী

- 0 প্রধানমন্ত্রী আজ সংসদে ভাষণ দিবেন রাজনীতি
- 1 বাংলাদেশ ভারতের বিপক্ষে ২০০ রান করেছে খেলা
- 2 শেয়ার বাজারে আজ ৫০ পয়েন্ট বৃদ্ধি অর্থনীতি
- 3 সরকারি নতুন নীতি ঘোষণা করা হয়েছে রাজনীতি
- 4 ঢাকার বিপক্ষে চট্টগ্রামের দুর্দান্ত পারফরমেন্স খেলা

SVM (linear kernel): নির্ভুলতা = 0.833

SVM (rbf kernel): নির্ভুলতা = 0.667

SVM (poly kernel): নির্ভুলতা = 0.500

সেরা কার্নেল: linear

বিস্তারিত রিপোর্ট